

```
(let ((greetings (ht-create)))
  (ht-empty? greetings))

t
```

The equality check can be used on a couple of hash tables to verify if they are the same, using the *ht-equal?* function as illustrated below:

```
(ht-equal? hash-table1 hash-table2) ;; Syntax

(let ((english (ht-create))
      (numbers (ht-create)))
  (ht-set! english "a" "A")
  (ht-set! numbers "1" "One")
  (ht-equal? english numbers))

nil
```

A few of the *ht.el* library functions accept a function as an argument and apply it to the items of the list. For example, the *ht-map* function takes a function with a key and value as arguments, and applies the function to each item in the hash table. For example:

```
(ht-map function hash-table) ;; Syntax

(let ((numbers (ht-create)))
  (ht-set! numbers 1 "One")
  (ht-map (lambda (x y) (* x 2)) numbers))

(2)
```

You can also use the *ht-each* API to iterate through each item in the hash-table. In the following example, the sum of all the values is calculated and finally printed in the output.

```
(ht-each function hash-table) ;; Syntax

(let ((numbers (ht-create))
      (sum 0))
  (ht-set! numbers "A" 1)
  (ht-set! numbers "B" 2)
  (ht-set! numbers "C" 3)
  (ht-each (lambda (key value) (setq sum (+ sum value)))
    numbers)
  (print sum))

6
```

The *ht-select* function can be used to match and pick a specific set of items in the list. For example:

```
(ht-select function hash-table) ;; Syntax
```

```
(let ((numbers (ht-create)))
  (ht-set! numbers 1 "One")
  (ht-set! numbers 2 "Two")
  (ht-items (ht-select (lambda (x y) (= x 2)) numbers)))

(("2" "Two"))
```

You can also reject a set of values by passing a filter function to the *ht-reject* API, and retrieve those items from the hash table that do not match the predicate function. In the following example, key 2 is rejected and the item with key 1 is returned.

```
(ht-reject function hash-table) ;; Syntax

(let ((numbers (ht-create)))
  (ht-set! numbers 1 "One")
  (ht-set! numbers 2 "Two")
  (ht-items (ht-reject (lambda (x y) (= x 2)) numbers)))

(("1" "One"))
```

If you want to mutate the existing hash table and remove the items that match a filter function, you can use the *ht-reject!* function as shown below:

```
(ht-reject! function hash-table) ;; Syntax

(let ((numbers (ht-create)))
  (ht-set! numbers 1 "One")
  (ht-set! numbers 2 "Two")
  (ht-reject! (lambda (x y) (= x 2)) numbers)
  (ht-items numbers))

(("1" "One"))
```

The *ht-find* function accepts a function and a hash table, and returns the items that satisfy the input function. For example:

```
(ht-find function hash-table) ;; Syntax

(let ((numbers (ht-create)))
  (ht-set! numbers 1 "One")
  (ht-set! numbers 2 "Two")
  (ht-find (lambda (x y) (= x 2)) numbers))

(2 "Two")
```

You can retrieve the items in the hash table using a specific set of keys with the *ht-select-keys* API, as illustrated below:

```
(ht-select-keys hash-table keys) ;; Syntax

(let ((numbers (ht-create)))
```